



PROJECT ASSIGNMENT

Empirical Project: SSH Security Weakness Study

Course: Cybersecurity

Kerala, India

01-07-2026

Varghese Kuruvilla

Enroll id: 35250473

Table of Content

Abstract.....	1
1.Introduction	1
2. Methodology.....	2
3. Experiment.....	4
4. Results	8
5. Discussion.....	9
6. Recommendations	10
7.Conclusion.....	10
8.References.....	10

Abstract

The project evaluates mainly used restrains which happens in SSH services to mitigate against the brute force attacks happens using the docker container environment. The main features examine the password based on three methods which are weak password without protection, strong password without protection and a strong password using fail2ban protection this showcase the password strength and defensive mechanism on authenticated security. This project results in showing the passwords which are weak are very highly vulnerable easily attacked by attackers whereas, stronger passwords provide little more reliable but it can be breakable using different methods but have more resistance compare to weak passwords. Strong passwords using fail2ban protection will help to reduce the effectiveness of multiple logins to same credentials and bans if the password is failed after a threshold attempts. The SSH services use key-based authentication, strong credential policies, and adaptive blocking as layered defences for making the services more secure and stable.

1.Introduction

SSH is secure and a common protocol which is mainly used by entire administrations for secure login and also used for credential authentication and remote management. SSH services mostly use the method or the common method is using password credentials and this method can help easily increase the chance of brute force attacks. In a brute force attacks the attackers uses multiple username and multiple password combination until the attacker able to login. In brute force attacks, if the password easy or weak the attackers can easily crack the account and able to login to the account. The report mainly investigate on SSH resilience against the password strength and unauthorized security controls happens in a brute force attack. In this method we are evaluating SSH using the strong password and using rate limiting protection which is fail2ban, which restricts user or attacker after multiple failed logins.

The testing of the different methods was or on a safe, local and authorized docker environment. The three methods for this experiment were using weak password with no protection, the other method was strong password with no protection and the last method was using strong passwords enabled with fail2ban inorder to check the limitation after unauthorized attempts. All these three configurations under the same environment demonstrates how the password strength and rate limiting mechanism to improves the security of SSH from brute force attacks.

2. Methodology

The SSH was run on docker container and using localhost 2222. The main reason for creating docker environment is to have a separate or local environment where all the testing, authentication does not affect any other services and it will help or does not contact any other host, external system or network systems to get affected by brute force attacks.

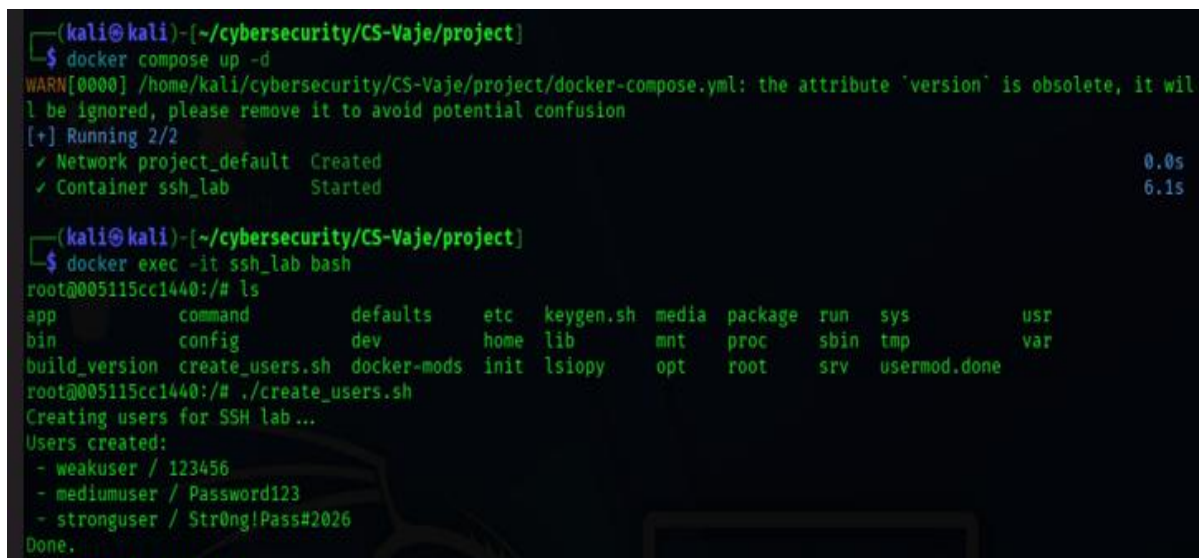
The laboratory started by running the command:

```
docker compose up -d
```

After running the above command, created a user account using the container:

```
docker exec -it ssh_lab bash
```

The user creation script was executed directly, without making any permission changes. After the user creation script executed. The script automatically made 3 test account which are weak user, medium user and strong user.



```
(kali@kali)-[~/cybersecurity/CS-Vaje/project]
$ docker compose up -d
WARN[0000] /home/kali/cybersecurity/CS-Vaje/project/docker-compose.yml: the attribute 'version' is obsolete, it will
be ignored, please remove it to avoid potential confusion
[+] Running 2/2
 ✓ Network project_default Created                                0.0s
 ✓ Container ssh_lab Started                                       6.1s

(kali@kali)-[~/cybersecurity/CS-Vaje/project]
$ docker exec -it ssh_lab bash
root@005115cc1440:/# ls
app      command  defaults  etc  keygen.sh  media  package  run  sys      usr
bin      config   dev       home lib        mnt     proc     sbin  tmp      var
build_version  create_users.sh  docker-mods  init  lsiopy    opt    root     srv   usermod.done
root@005115cc1440:/# ./create_users.sh
Creating users for SSH lab...
Users created:
- weakuser / 123456
- mediumuser / Password123
- stronguser / Str0ng!Pass#2026
Done.
```

Each account is configured with different passwords security levels, after running the script in environment the accounts were checked whether they are created successfully. The command Nmap was used to check whether the secure service was running on the local host 2222 using the command:

```
nmap -sS -sV -p 2222 localhost
```

Using this scan confirmed that the SSH was running on the host 2222 and also accepting connection to localhost 2222. After checking the SSH server was performing correctly the next step was to run hydra which is used to password based brute force attacks on each test accounts. A password .txt file was given to hydra which contains some password list. The below command was used to test the accounts.

```
hydra -l weakuser -P passwords.txt ssh://localhost:2222
```

```
hydra -l mediumuser -P passwords.txt ssh://localhost:2222
```

```
hydra -l stronguser -P passwords.txt ssh://localhost:2222
```

Each attack was attempted to login using the password list until correct password is found or list is exhausted. During every test it is used to record the number password attempt, able to login, total time taken for a brute force attack. These methods were used for all the accounts in the experiment.

Fail2Ban configuration was used to protect the passwords from brute force attacks. Fail2Ban enabled in the SSH container which is used to monitor the password attempts and blocks the IP address after several attempts and protect from attacks. This help to reduce the attackers from trying to do password guessing.

The following SSH jail configuration was used:

```
GNU nano 9.0 /etc/fail2ban/jail.local
[[docker-ssh]
enabled = true
filter = docker-ssh
port = 2222

logpath = /home/kali/cybersecurity/CS-Vaje/project/config/logs/openssh/current

backend = auto
maxretry = 3
findtime = 60
bantime = 300

ignoreself = false
ignoreip = 127.0.0.1/8 ::1

action = iptables[name=docker-ssh, port=2222, protocol=tcp]
```

With the above configuration bantime was after 300 seconds and maximum try is 3 times within a 60 second period. These results were used to compare with previous tests to check how rate limiting affected the speed and possibility of the brute-force attack.

3. Experiment

User creation

After the docker was started running in the environment, the user creation was executed directly without any permission needed. After running the user creation script, three test accounts were created which are weakuser, mediumuser, stronguser which is having different password strength. After the account created it was ensured that test accounts created successfully. This successfully help us to understand the docker runs perfectly in the local host 2222 and in the safe environment.

```
(kali@kali) [~/cybersecurity/CS-Vaje/project]
$ docker compose up -d
WARN[0000] /home/kali/cybersecurity/CS-Vaje/project/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 2/2
  ✓ Network project_default    Created                                0.0s
  ✓ Container ssh_lab          Started                             6.1s

(kali@kali) [~/cybersecurity/CS-Vaje/project]
$ docker exec -it ssh_lab bash
root@005115cc1440:/# ls
app          command     defaults    etc          keygen.sh   media       package     run          sys          usr
bin          config      dev         home         lib         mnt         proc       sbin        tmp          var
build_version create_users.sh docker-mods init         lsiopy      opt         root       srv          usermod.done
root@005115cc1440:/# ./create_users.sh
Creating users for SSH lab...
Users created:
- weakuser / 123456
- mediumuser / Password123
- stronguser / Str0ng!Pass#2026
Done.
root@005115cc1440:/# ls
app          command     defaults    etc          keygen.sh   media       package     run          sys          usr
bin          config      dev         home         lib         mnt         proc       sbin        tmp          var
build_version create_users.sh docker-mods init         lsiopy      opt         root       srv          usermod.done
root@005115cc1440:/# cat /etc/passwd
root:x:0:0:root:/root:/bin/sh
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21::/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:1000:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/:/sbin/nologin
klogd:x:100:101:klogd:/dev/null:/sbin/nologin
student:x:1000:1000::/config:/bin/bash
weakuser:x:1001:1001::/home/weakuser:/bin/ash
mediumuser:x:1002:1002::/home/mediumuser:/bin/ash
stronguser:x:1003:1003::/home/stronguser:/bin/ash
root@005115cc1440:/#
```

Reconnaissance

The phase Reconnaissance is usually used to check the service port which is running and the correct which is needed for the brute force attack. To check this we usually use Nmap tool which is used to scan tcp port 2222 and tried to find out whether it is running on the same port. The scan confirms that port is running on a OpenSSH. This usually verifies the server is available, reachable and connection for the authentication to work on.

```
(kali@kali)-[~/cybersecurity/CS-Vaje/project]
└─$ nmap -sS -sV -p 2222 localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 14:49 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00013s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
2222/tcp  open  ssh      OpenSSH 10.2 (protocol 2.0)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.49 seconds
```

Brute-Force Testing

Hydra is tool which used for performing brute force attacks. This tool was used to perform on all three test accounts which are weakuser, mediumuser and stronguser using the same password list from password.txt. Hydra mainly used to track the time taken to crack the password, number of attempts occurred to get correct credentials.

Weakuser:

```
(kali@kali)-[~/cybersecurity/CS-Vaje/project]
└─$ hydra -l weakuser -P passwords.txt ssh://localhost:2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 14:46:03
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 20 login tries (l:1/p:20), ~2 tries per task
[DATA] attacking ssh://localhost:2222/
[2222][ssh] host: localhost login: weakuser password: 123456
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 1 final worker threads did not complete until end.
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 14:46:03
```


Mediumuser:

```
(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
└─$ hydra -l mediumuser -P passwords.txt ssh://localhost:2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 16:22:46
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 22 login tries (l:1/p:22), ~2 tries per task
[DATA] attacking ssh://localhost:2222/
[2222][ssh] host: localhost login: mediumuser password: Password123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 16:22:47
```

Stronguser:

```
(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
└─$ hydra -l stronguser -P passwords.txt ssh://localhost:2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 16:24:06
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 22 login tries (l:1/p:22), ~2 tries per task
[DATA] attacking ssh://localhost:2222/
[2222][ssh] host: localhost login: stronguser password: Str0ng!Pass#2026
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 16:24:07
```

The above screenshot proves that all 3-test account was attacked under brute force and password was cracked this is due to because of using password.txt file which contains all the passwords in the list. This indicates that it was a dictionary attack as already passwords are available in the list. In a real brute force attack each test usually accessed number of successes, number of login and time taken to correct the credential and all these are taken under comparison and evaluate the data for the next attack.

Fail2Ban Protection Test

To find defensive method for protecting against the brute force attacks by enabling fail2ban in SSH server.

```
(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
└─$ sudo nano /etc/fail2ban/jail.d/sshd.local
[sudo] password for kali:

(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
└─$
```


After SSH and jail was configured same hydra attack were done on test account stronguser with same password.

```
(kali㉿kali)-[/etc/fail2ban]
$ cat jail.local
[docker-ssh]
enabled = true
filter = docker-ssh
port = 2222

logpath = /home/kali/cybersecurity/CS-Vaje/project/config/logs/openssh/current

backend = auto
maxretry = 3
findtime = 60
bantime = 300

ignoreself = false
ignoreip = 127.0.0.1/8 ::1

action = iptables[name=docker-ssh, port=2222, protocol=tcp]
```

```
(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
$ hydra -l stronguser -P passwords.txt ssh://localhost:2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-26 21:22:22
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 22 login tries (l:1/p:22), ~2 tries per task
[DATA] attacking ssh://localhost:2222/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-26 21:22:24
```

Unlike, the previous hydra was unable to correct password. After multiple password attempt fail2ban has banned the attacker's IP address. Thus, it result, hydra cannot no longer attack or crack the password with rate limiting and IP address banning.

Inorder, to verify the protection mechanism is enabled in the server the command is used as:

```
fail2ban-client status docker-ssh
```

The outcome of this command will verify that jail is active, SSH log is monitoring and attacker's IP address is added in the banned list. This will help us to understand that ptotection mechanism completely working successfully as it help to track the multiple authentication failure and also complete ban on attacker's IP address.

```
(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
$ sudo fail2ban-client set docker-ssh unbanip 172.22.0.1
1

(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
$ sudo fail2ban-client status docker-ssh
Status for the jail: docker-ssh
├ Filter
│ ├── Currently failed: 0
│ ├── Total failed: 50
│ └─ File list: /home/kali/cybersecurity/CS-Vaje/project/config/logs/openssh/current
└─ Actions
    ├── Currently banned: 0
    ├── Total banned: 3
    └─ Banned IP list:

(kali㉿kali)-[~/cybersecurity/CS-Vaje/project]
$ sudo fail2ban-client status docker-ssh
Status for the jail: docker-ssh
├ Filter
│ ├── Currently failed: 1
│ ├── Total failed: 72
│ └─ File list: /home/kali/cybersecurity/CS-Vaje/project/config/logs/openssh/current
└─ Actions
    ├── Currently banned: 1
    ├── Total banned: 4
    └─ Banned IP list: 172.22.0.1
```

4. Results

The result of this experiment shows the effect of password strength and defensive control on SSH authentication during digital dictionary.

In the first scenario, weakuser account uses the password “123456” using the hydra the attacker was able to crack the credential within 20 attempts as already password was in the list. This shows that weak and common passwords can be easily cracked as it will be already available in the attacker’s dictionary.

In the second scenario, mediumuser uses password “Password123” using the hydra the attacker was able to crack the credential within 22 attempts as already it was available in the list. In this case password was long and it was common password so it took little more time to crack the credentials.

In third scenario, the stronguser with no protect uses the password ‘StrOng!Pass#2026’ after use of hydra the password was able to crack after 22 attempts. In this case the password was more complex and lengthier. Even though, the password is complex the attackers can easily crack a password with no protection.

In last scenario, the stronguser using Fail2Ban protection and using the same password .After a hydra attacks the password was not able to crack and attacker’s ip address got banned after

multiple attempt. As, a result of this hydra cannot able to find the credentials as the attacker got banned on the IP address.

Overall, checking the experiment it was found that for three passwords with no protection was able to crack the credential by attackers. The attacker was able to crack the password as it was already present in the list. But in real a brute force attackers will be predicting and they can easily crack the strong password with no protection. The fail2ban protection makes understand that it doesn't depend on the password strength it focuses on number of failed attempts and time taken by attacker and hence, resulting in banning IP address of attacker.

5. Discussion

In this experiment helped to realize that strong password does not provide any protection to account in OpenSSH server shell in a dictionary attack as already the password list is available and the attackers can easily guess the password. In all three cases the weak, medium and strong user with no protection attacker was able to find the password which were listed. It helps us to understand using hydra the strong password which are having complex letters can be cracked as it can be predictable and easy to find.

This experiment also highlight difference between the brute force attacks and dictionary attacks. A brute force attacks is in which attackers use trial and error method and try to find out the credential. It will take more time and its more time consuming and complex and attackers has to compute until the password is found. Whereas, in the case of dictionary attack already the passwords which are already available in common list and the attackers try the passwords from the list and find the match.

This experiment also shows that along with complex passwords it should have protection method fail2ban in SSH server which can resist that attackers from cracking the password. Unlike, the previous experiments the hydra attempts failed after multiple attempts and also got banned ip address of attackers from attacking. This would help to reduce the attempts from attackers from the same source but it can be less efficient if the attacker user multiple ip address.

6. Recommendations

The following methods can be used to improve protection from brute force attacks are:

- Enable multi factor authentication to add an extra layer protection to the passwords.
- Change the default SSH ports to reduce automated scanning and also attacks attempts from bots.
- Ensure the using SSH server should be updated regularly inorder to protect from new vulnerability.
- Removing unused account or delete the accounts which are not using can helps to reduce the attacks.
- Adding rate limiting attempts or fail2ban which can restrict attackers from multiple tries.

7. Conclusion

The experiment demonstrates that SSH accounts are vulnerable to dictionary attacks when the correct password is available in the list. For the account weak, medium and strong the passwords were able to crack because there was no protection enabled for the SSH. Thus, it helps understand complexity along cannot prevent attacks it should be provided some protection mechanism.

Implementing fail2ban protection to SSH has helps to understand that the password which are complex can be protected as they will track multiple attempts and blocks the ip address of the attacker due to failed authentication. Overall, password complexity and protection cannot secure the password it should have some other methods like MFA, regular monitoring of the server to improve the safety of the accounts from the attackers.

8. References

Kali Linux. (n.d.). *Hydra*. <https://www.kali.org/tools/hydra/>

LabEx. (n.d.). *Brute-force SSH in Hydra*. <https://labex.io/tutorials/hydra-brute-force-ssh-in-hydra-549926>

SSH Communications Security. (n.d.). *Types of password attacks and how to prevent them*. <https://www.ssh.com/academy/types-of-password-attacks-and-how-to-prevent-them>

Recorded Future. (n.d.). *Nmap commands: A guide to network scanning*.
<https://www.recordedfuture.com/threat-intelligence-101/tools-and-techniques/nmap-commands>

McGuire, M. J. (2023, November 3). *Using Fail2Ban on a Docker host for SSH containers*.
<https://blog.mikejmcguire.com/2023/11/03/using-fail2ban-on-a-docker-host-for-ssh-containers/>

DigitalOcean. (n.d.). *How to protect SSH with Fail2Ban on Ubuntu 20.04*.
<https://www.digitalocean.com/community/tutorials/how-to-protect-ssh-with-fail2ban-on-ubuntu-20-04>

Linode. (n.d.). *Using Fail2Ban to secure your server: A tutorial*.
<https://www.linode.com/docs/guides/using-fail2ban-to-secure-your-server-a-tutorial/>